

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



OPERATING SYSTEMS

AAT REPORT

on

FILE ALLOCATION, FILE ORGANISATION AND DISK SCHEDULING ALGORITHMS

Submitted by

YASH KUMAR (1BM24CS336)

Under the Guidance of

Rajeshwari Madli

Assistant Professor, BMSCE

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**OPERATING SYSTEMS – 23CS4PCOPS**” carried out by **YASH KUMAR (1BM24CS336)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of **OPERATING SYSTEMS - (23CS4PCOPS)** work prescribed for the said degree.

Rajeshwari Madli
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

INDEX

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turn around time and waiting time. a) FCFS b) SJF c) Round-Robin d) Priority	5 – 20
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	21 – 23
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms a) Rate- Monotonic b) Proportional Share	24 - 30
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	31 – 35
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance.	36 – 38
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	39 – 43
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	44 – 50

Course Outcome

CO1	Apply the different concepts and functionalities of Operating System
CO2	Analyze various Operating system strategies and techniques
CO3	Demonstrate the different functionalities of Operating System
CO4	Conduct practical experiments to implement the functionalities of Operating system

1. Write a C program to simulate the following CPU scheduling algorithm to find turn around time and waiting time.

a)FCFS

```
#include <stdio.h>

#define MAX_PROCESSES 100

typedef struct {
    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
    int response_time;
} Process;

static void sort_by_arrival(Process *procs, int n)
{
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - 1 - i; ++j) {
            if (procs[j].arrival_time > procs[j+1].arrival_time) {
                Process tmp = procs[j];
                procs[j] = procs[j+1];
                procs[j+1] = tmp;
            }
        }
    }
}

static void calculate_times(Process *procs, int n)
{
    int current_time = 0;
    double total_waiting = 0.0;
    double total_turnaround = 0.0;
    for (int i = 0; i < n; ++i) {
        if (current_time < procs[i].arrival_time)
            current_time = procs[i].arrival_time;
        procs[i].response_time = current_time - procs[i].arrival_time;
        procs[i].waiting_time = procs[i].response_time;
        procs[i].completion_time = current_time + procs[i].burst_time;
        procs[i].turnaround_time = procs[i].completion_time - procs[i].arrival_time;
        current_time = procs[i].completion_time;
        total_waiting += procs[i].waiting_time;
        total_turnaround += procs[i].turnaround_time;
    }
    printf("\n%-5s %-10s %-7s %-7s %-12s %-10s %-12s %-10s\n", "PID", "Arrival", "Burst", "Start",
"Completion", "Waiting", "Turnaround", "Response");
    printf("=====\n");
    for (int i = 0; i < n; ++i) {
        int start_time = procs[i].completion_time - procs[i].burst_time;
        printf("%-5d %-10d %-7d %-7d %-12d %-10d %-12d %-10d\n",
            procs[i].pid,
            procs[i].arrival_time,
            procs[i].burst_time,
```

```

        start_time,
        procs[i].completion_time,
        procs[i].waiting_time,
        procs[i].turnaround_time,
        procs[i].response_time);
    }
    printf("\nAverage Waiting Time   = %.2f\n", total_waiting / n);
    printf("Average Turnaround Time = %.2f\n", total_turnaround / n);
}
int main(void)
{
    int n;
    Process procs[MAX_PROCESSES];
    printf("Enter number of processes: ");
    if (scanf("%d", &n) != 1 || n <= 0 || n > MAX_PROCESSES) {
        fprintf(stderr, "Invalid number of processes (1-%d)\n", MAX_PROCESSES);
        return 1;
    }
    for (int i = 0; i < n; ++i) {
        procs[i].pid = i + 1;
        printf("Enter arrival time and burst time for process %d: ", procs[i].pid);
        if (scanf("%d %d", &procs[i].arrival_time, &procs[i].burst_time) != 2) {
            fprintf(stderr, "Invalid input\n");
            return 1;
        }
    }
    sort_by_arrival(procs, n);
    calculate_times(procs, n);
    return 0;
}

```

```

Enter number of processes: 4
Enter arrival time and burst time for process 1: 0 7
Enter arrival time and burst time for process 2: 8 3
Enter arrival time and burst time for process 3: 3 4
Enter arrival time and burst time for process 4: 5 6

PID  Arrival  Burst  Start  Completion  Waiting  Turnaround  Response
=====
1    0         7      0       7           0         7           0
3    3         4      7      11           4         8           4
4    5         6     11     17           6        12           6
2    8         3     17     20           9        12           9

Average Waiting Time   = 4.75
Average Turnaround Time = 9.75

```

b)SJF (Pre-Emptive and Non-Pre-Emptive)
i.Non Pre-Emptive

```
#include <stdio.h>
struct Process {
    int pid, at, bt, ct, tat, wt, completed;
};
int main() {
    int n, time = 0, completed = 0, min_bt, index;
    float total_tat = 0, total_wt = 0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    struct Process p[n];
    for(int i=0;i<n;i++){
        printf("Process %d Arrival Time: ",i+1);
        scanf("%d",&p[i].at);
        printf("Process %d Burst Time: ",i+1);
        scanf("%d",&p[i].bt);

        p[i].pid=i+1;
        p[i].completed=0;
    }
    while(completed!=n){
        min_bt=9999;
        index=-1;
        for(int i=0;i<n;i++){
            if(p[i].at<=time && p[i].completed==0){
                if(p[i].bt<min_bt){
                    min_bt=p[i].bt;
                    index=i;
                }
            }
        }
        if(index!=-1){
            time+=p[index].bt;
            p[index].ct=time;
            p[index].tat=p[index].ct-p[index].at;
            p[index].wt=p[index].tat-p[index].bt;
            total_tat+=p[index].tat;
            total_wt+=p[index].wt;
            p[index].completed=1;
            completed++;
        }
        else
            time++;
    }
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(int i=0;i<n;i++)
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid,p[i].at,p[i].bt,p[i].ct,p[i].tat,p[i].wt);
    printf("\nAverage Turnaround Time = %.2f",total_tat/n);
    printf("\nAverage Waiting Time = %.2f",total_wt/n);
}
```

```

Enter number of processes: 5
Process 1 Arrival Time: 0
Process 1 Burst Time: 3
Process 2 Arrival Time: 2
Process 2 Burst Time: 2
Process 3 Arrival Time: 3
Process 3 Burst Time: 5
Process 4 Arrival Time: 4
Process 4 Burst Time: 4
Process 5 Arrival Time: 6
Process 5 Burst Time: 1

PID    AT    BT    CT    TAT    WT
1       0     3     3     3     0
2       2     2     5     3     1
3       3     5    15    12     7
4       4     4     9     5     1
5       6     1    10     4     3

Average Turnaround Time = 5.40
Average Waiting Time = 2.40
Process returned 0 (0x0)   execution time : 20.849 s
Press any key to continue.

```

ii.Pre-Emptive

```

#include <stdio.h>
#include <stdbool.h>
struct Process {
    int pid,at,bt,rt,ct,tat,wt,resp_t;
    bool started;
};
int main() {
    int n, time = 0, completed = 0;
    float total_tat = 0, total_wt = 0, total_resp_t = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("\nProcess %d:\n", p[i].pid);
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        p[i].rt = p[i].bt;
        p[i].started = false;
    }
    while (completed < n) {
        int index = -1;
        int min_rt = 9999;
        for (int i = 0; i < n; i++) {
            if (p[i].at <= time && p[i].rt > 0 && p[i].rt < min_rt) {
                min_rt = p[i].rt;
                index = i;
            }
        }
        if (index != -1) {
            if (!p[index].started) {

```



```

        p[index].resp_t = time - p[index].at;
        total_resp_t += p[index].resp_t;
        p[index].started = true;
    }
    p[index].rt--;
    time++;
    if (p[index].rt == 0) {
        completed++;
        p[index].ct = time;
        p[index].tat = p[index].ct - p[index].at;
        p[index].wt = p[index].tat - p[index].bt;
        total_tat += p[index].tat;
        total_wt += p[index].wt;
    }
} else {
    time++;
}
}
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt, p[i].resp_t);
}
printf("\nAverage Turnaround Time = %.2f", total_tat / n);
printf("\nAverage Waiting Time = %.2f", total_wt / n);
printf("\nAverage Response Time = %.2f\n", total_resp_t / n);
return 0;
}

```

```

Enter number of processes: 5

Process 1:
Arrival Time: 0
Burst Time: 3

Process 2:
Arrival Time: 2
Burst Time: 2

Process 3:
Arrival Time: 3
Burst Time: 5

Process 4:
Arrival Time: 4
Burst Time: 4

Process 5:
Arrival Time: 6
Burst Time: 1

PID    AT    BT    CT    TAT    WT    RT
1       0     3     3     3     0     0
2       2     2     5     3     1     1
3       3     5    15    12     7     7
4       4     4    10     6     2     1
5       6     1     7     1     0     0

Average Turnaround Time = 5.00
Average Waiting Time = 2.00
Average Response Time = 1.80

```

c)Round-Robin

```
#include <stdio.h>
#include <string.h>
#define MAX 10
#define QUEUE_SIZE (MAX * 10)
typedef struct {
    int pid,at,bt,rt,ct,tat,wt;
} Process;
typedef struct {
    int data[QUEUE_SIZE];
    int front;
    int rear;
} Queue;
void initQueue(Queue *q) {
    q->front = 0;
    q->rear = 0;
}
int isEmpty(Queue *q) {
    return q->front == q->rear;
}
void enqueue(Queue *q, int val) {
    q->data[q->rear++] = val;
}
int dequeue(Queue *q) {
    return q->data[q->front++];
}
void addArrivals(Process procs[], int n, Queue *q, int inQueue[], int currentTime) {
    for (int i = 0; i < n; i++) {
        if (!inQueue[i] && procs[i].rt > 0 && procs[i].at <= currentTime) {
            enqueue(q, i);
            inQueue[i] = 1;
        }
    }
}
int main() {
    Process procs[MAX];
    Queue readyQueue;
    int inQueue[MAX];
    int n, tq;
    int completed = 0, time = 0;
    printf("=== Round Robin CPU Scheduling ===\n\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        procs[i].pid = i + 1;
        printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
        scanf("%d %d", &procs[i].at, &procs[i].bt);
        procs[i].rt = procs[i].bt;
    }
    printf("Enter Time Quantum: ");
    scanf("%d", &tq);
    initQueue(&readyQueue);
    memset(inQueue, 0, sizeof(inQueue));
    addArrivals(procs, n, &readyQueue, inQueue, time);
    printf("\nGantt Chart:\n|");
```

```

while (completed < n) {
    if (isEmpty(&readyQueue)) {
        time++;
        addArrivals(procs, n, &readyQueue, inQueue, time);
        continue;
    }
    int idx = dequeue(&readyQueue);
    Process *p = &procs[idx];
    if (p->rt > tq) {
        printf(" P%d |", p->pid);
        time += tq;
        p->rt -= tq;
        addArrivals(procs, n, &readyQueue, inQueue, time);
        enqueue(&readyQueue, idx);
    } else {
        printf(" P%d |", p->pid);
        time += p->rt;
        p->rt = 0;
        p->ct = time;
        completed++;
        addArrivals(procs, n, &readyQueue, inQueue, time);
    }
}
printf("\n\n");
float totalTAT = 0, totalWT = 0;
printf("%-8s %-8s %-8s %-8s %-8s %-8s\n",
    "Process", "AT", "BT", "CT", "TAT", "WT");
printf("-----\n");
for (int i = 0; i < n; i++) {
    procs[i].tat = procs[i].ct - procs[i].at;
    procs[i].wt = procs[i].tat - procs[i].bt;
    totalTAT += procs[i].tat;
    totalWT += procs[i].wt;
    printf("P%-7d %-8d %-8d %-8d %-8d %-8d\n",
        procs[i].pid, procs[i].at, procs[i].bt,
        procs[i].ct, procs[i].tat, procs[i].wt);
}
printf("-----\n");
printf("Average Turnaround Time : %.2f\n", totalTAT / n);
printf("Average Waiting Time : %.2f\n", totalWT / n);
return 0;
}

```

```

=== Round Robin CPU Scheduling ===

Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 10
Enter Arrival Time and Burst Time for P2: 2 6
Enter Arrival Time and Burst Time for P3: 4 3
Enter Arrival Time and Burst Time for P4: 6 8
Enter Time Quantum: 2

Gantt Chart:
| P1 | P2 | P1 | P3 | P2 | P4 | P1 | P3 | P2 | P4 | P1 | P4 | P1 | P4 |

Process  AT      BT      CT      TAT      WT
-----
P1       0       10      25       25       15
P2       2        6      17       15        9
P3       4        3      15       11        8
P4       6        8      27       21       13
-----

Average Turnaround Time : 18.00
Average Waiting Time : 11.25

```

d)Priority (Pre-Emptive and Non-Pre-Emptive)
i.Non-Pre-Emptive

```
#include <stdio.h>
struct Process {
    int pid, at, bt, pr, ct, tat, wt, rt, completed;
};
int main() {
    int n, time=0, completed=0, highest_pr, index;
    float avg_tat=0, avg_wt=0, avg_rt=0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    struct Process p[n];
    for(int i=0;i<n;i++){
        printf("\nProcess %d:\n", i+1);
        printf("Arrival Time: ");
        scanf("%d",&p[i].at);
        printf("Burst Time: ");
        scanf("%d",&p[i].bt);
        printf("Priority: ");
        scanf("%d",&p[i].pr);
        p[i].pid=i+1;
        p[i].completed=0;
    }
    while(completed<n){
        highest_pr=9999;
        index=-1;
        for(int i=0;i<n;i++){
            if(p[i].at<=time && p[i].completed==0){
                if(p[i].pr<highest_pr){
                    highest_pr=p[i].pr;
                    index=i;
                }
            }
        }
        if(index!=-1){
            p[index].rt = time - p[index].at;
            time+=p[index].bt;
            p[index].ct=time;
            p[index].tat=p[index].ct-p[index].at;
            p[index].wt=p[index].tat-p[index].bt;
            avg_tat+=p[index].tat;
            avg_wt+=p[index].wt;
            avg_rt+=p[index].rt;
            p[index].completed=1;
            completed++;
        }
        else {
            time++;
        }
    }
    printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");
    for(int i=0;i<n;i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid,p[i].at,p[i].bt,p[i].pr,p[i].ct,p[i].tat,p[i].wt,p[i].rt);
    }
}
```

```

}
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Response Time = %.2f\n",avg_rt/n);
return 0;
}

```

```

Enter number of processes: 5

Process 1:
Arrival Time: 0
Burst Time: 3
Priority: 5

Process 2:
Arrival Time: 2
Burst Time: 2
Priority: 3

Process 3:
Arrival Time: 3
Burst Time: 5
Priority: 2

Process 4:
Arrival Time: 4
Burst Time: 4
Priority: 4

Process 5:
Arrival Time: 6
Burst Time: 1
Priority: 1

PID   AT   BT   PR   CT   TAT   WT   RT
1      0    3    5    3    3     0    0
2      2    2    3   11    9     7    7
3      3    5    2    8    5     0    0
4      4    4    4   15   11     7    7
5      6    1    1    9    3     2    2

Average Turnaround Time = 6.20
Average Waiting Time = 3.20
Average Response Time = 3.20

```

ii.Pre-Emptive

```

#include <stdio.h>
#include <stdbool.h>
struct Process {
    int pid, at, bt, pr, rt, ct, tat, wt, resp_t;
    bool started;
};
int main() {
    int n, time = 0, completed = 0;
    float total_tat = 0, total_wt = 0, total_resp_t = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("\nProcess %d:\nArrival Time: ", p[i].pid);
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Priority: ");
        scanf("%d", &p[i].pr);
        p[i].rt = p[i].bt;
        p[i].started = false;
    }
    while (completed < n) {
        int index = -1;

```

```

int highest_pr = 9999;
for (int i = 0; i < n; i++) {
    if (p[i].at <= time && p[i].rt > 0 && p[i].pr < highest_pr) {
        highest_pr = p[i].pr;
        index = i;
    }
}
if (index != -1) {
    if (!p[index].started) {
        p[index].resp_t = time - p[index].at;
        total_resp_t += p[index].resp_t;
        p[index].started = true;
    }
    p[index].rt--;
    time++;
    if (p[index].rt == 0) {
        completed++;
        p[index].ct = time;
        p[index].tat = p[index].ct - p[index].at;
        p[index].wt = p[index].tat - p[index].bt;
        total_tat += p[index].tat;
        total_wt += p[index].wt;
    }
} else {
    time++;
}
}
printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].pr, p[i].ct, p[i].tat, p[i].wt, p[i].resp_t);
}
printf("\nAverage Turnaround Time = %.2f", total_tat / n);
printf("\nAverage Waiting Time = %.2f", total_wt / n);
printf("\nAverage Response Time = %.2f\n", total_resp_t / n);
return 0;
}

```

```

Enter number of processes: 5

Process 1:
Arrival Time: 0
Burst Time: 3
Priority: 5

Process 2:
Arrival Time: 2
Burst Time: 2
Priority: 3

Process 3:
Arrival Time: 3
Burst Time: 5
Priority: 2

Process 4:
Arrival Time: 4
Burst Time: 4
Priority: 4

Process 5:
Arrival Time: 6
Burst Time: 1
Priority: 1

PID    AT    BT    PR    CT    TAT    WT    RT
1      0      3      5     15     15     12      0
2      2      2      3     10      8      6      0
3      3      5      2      9      6      1      0
4      4      4      4     14     10      6      6
5      6      1      1      7      1      0      0

Average Turnaround Time = 8.00
Average Waiting Time = 5.00
Average Response Time = 1.20

```

2/3/20 #LAB-2

Q. Write a C program to simulate the following CPU scheduling algorithms to find the average turnaround time and waiting time.

a) FCFS

```

#include <stdio.h>
#define MAX_PROCESSES 100
typedef struct {
    int pid; int arrival_time;
    int burst_time; int waiting_time;
    int turnaround_time; int completion_time;
    int response_time;
} Process;

static void sort_by_arrival(Process *procs, int n) {
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - 1 - i; ++j) {
            if (procs[j].arrival_time > procs[j+1].arrival_time) {
                Process tmp = procs[j];
                procs[j] = procs[j+1];
                procs[j+1] = tmp;
            }
        }
    }
}

static void calc_times(Process *procs, int n) {
    int current_time = 0;
    double total_waiting = 0.0, total_turnaround = 0.0;
    for (int i = 0; i < n; ++i) {
        if (current_time < procs[i].arrival_time)
            current_time = procs[i].arrival_time;
        procs[i].response_time = current_time - procs[i].arrival_time;
        procs[i].waiting_time = procs[i].response_time;
        procs[i].completion_time = current_time + procs[i].burst_time;
        procs[i].turnaround_time = procs[i].completion_time - procs[i].arrival_time;
        current_time = procs[i].completion_time;
        total_waiting += procs[i].waiting_time;
        total_turnaround += procs[i].turnaround_time;
    }

    printf("\n%-5s %-10s %-7s %-7s %-12s %-10s %-10s\n",
        "PID", "Arrival", "Burst", "Start", "Completion", "Waiting",
        "Turnaround", "Response");
    printf("\n");
    for (int i = 0; i < n; ++i) {
        int start_time = procs[i].completion_time - procs[i].burst_time;
        printf("%-5d %-10d %-7d %-7d %-12d %-10d %-10d\n",
            procs[i].pid,
            procs[i].arrival_time,
            procs[i].burst_time,
            start_time,
            procs[i].completion_time,
            procs[i].waiting_time,
            procs[i].turnaround_time);
    }
    printf("\n Average Waiting Time = %.2f\n",

```


→ Output

Enter number of processes: 5

Process 1 Arrival Time: 0

Process 1 Burst Time: 3

```

1 2 11 : 2
2 2 11 : 2
3 3 11 : 3
4 3 11 : 5
5 4 11 : 4
6 4 11 : 4
7 5 11 : 6
8 5 11 : 1
    
```

PID	AT	BT	CT	TAT	WT
1	0	3	3	3	0
2	2	2	5	3	1
3	3	5	15	12	7
4	4	4	9	5	1
5	6	1	10	4	3

Average Turnaround Time = 5.40

Average Waiting Time = 2.40

- Verification

GANIT Chart

P ₁	P ₂	P ₄	P ₅	P ₃
0	3	5	9	10
				15

Q2. WACP to simulate SJF scheduling algorithm per cursive

```

#include <stdio.h>
#include <stdlib.h>
int main() {
    int n, time = 0, completed = 0, min, index;
    int at[10], bt[10];
    struct Process {
        int pid, at, bt, wt, ct, tat, rt, resp_t;
        bool started;
    };
    int main() {
        int n, time = 0, completed = 0;
        float total_tat = 0, total_wt = 0, total_resp_t = 0;
        printf("Enter number of processes: ");
        scanf("%d", &n);
        struct Process p[10];
        for (int i = 0; i < n; i++) {
            p[i].pid = i + 1;
            printf("\nProcess %d: \n", p[i].pid);
            printf("Arrival Time: ");
            scanf("%d", &p[i].at);
            printf("Burst Time: ");
            scanf("%d", &p[i].bt);
            p[i].wt = p[i].bt;
            p[i].started = false;
        }
        while (completed < n) {
            int index = -1;
            int min_bt = 9999;
            for (int i = 0; i < n; i++) {
                if (p[i].at <= time && p[i].wt < min_bt && !p[i].started) {
                    min_bt = p[i].wt;
                    index = i;
                }
            }
            if (index != -1) {
                p[index].resp_t = time - p[index].at;
                total_resp_t += p[index].resp_t;
                p[index].started = true;
                p[index].wt--;
                time++;
                if (p[index].wt == 0) {
                    completed++;
                    p[index].ct = time;
                    p[index].tat = p[index].ct - p[index].at;
                    p[index].wt = p[index].tat - p[index].bt;
                    total_tat += p[index].tat;
                    total_wt += p[index].wt;
                }
            } else {
                time++;
            }
        }
        printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
        for (int i = 0; i < n; i++) {
            printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
                p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt,
            );
        }
        printf("\nAverage Turnaround Time = %.2f", total_tat/n);
        printf("\nAverage Waiting Time = %.2f", total_wt/n);
        printf("\nAverage Response Time = %.2f", total_resp_t/n);
        return 0;
    }
}
    
```

→ Output

Enter number of processes: 5

Process 1

Arrival Time: 0

Burst Time: 3

Process 2:	3	4	5
	2	3	6
	2	5	4

PID	AT	BT	CT	TAT	WT	RT
1	0	3	3	3	0	0
2	2	2	5	3	1	1
3	3	5	15	12	7	7
4	4	4	10	6	2	1
5	6	1	7	1	0	0

Average Turnaround Time: 5.00

Average Waiting Time: 2.00

Average Response Time: 1.80

→ GANIT →

P ₁	P ₂	P ₄	P ₅	P ₃
0	3	5	6	7
				10
				15

Q3. WACP to simulate Priority Scheduling algorithm non-preemptive

```
#include <stdio.h>
struct Process {
    int pid, at, bt, ct, tat, wt, rt, completed;
};

int main() {
    int n, time = 0, completed = 0, highest_pn, index;
    float avg_tat = 0, avg_wt = 0, avg_rt = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        printf("Process %d: ", i+1);
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Priority: ");
        scanf("%d", &p[i].pn);
        p[i].pid = i+1;
        p[i].completed = 0;
    }

    while (completed < n) {
        highest_pn = 9999;
        index = -1;
        for (int i = 0; i < n; i++) {
            if (p[i].at <= time && p[i].completed == 0) {
                if (p[i].pn < highest_pn) {
                    highest_pn = p[i].pn;
                    index = i;
                }
            }
        }

        if (index != -1) {
            p[index].ct = time - p[index].at;
            time = p[index].bt;
            p[index].ct = time;
            p[index].tat = p[index].ct + p[index].at;
            p[index].wt = p[index].tat - p[index].bt;
            avg_tat += p[index].tat;
            avg_wt += p[index].wt;
            avg_rt += p[index].rt;
            p[index].completed = 1;
            completed++;
        } else {
            time++;
        }
    }

    printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt, p[i].pn, p[i].ct, p[i].tat, p[i].wt, p[i].rt);
    }

    printf("Average Turnaround Time = %.2f", avg_tat/n);
    printf("Average Waiting Time = %.2f", avg_wt/n);
    printf("Average Response Time = %.2f", avg_rt/n);
}
```

highest_pn = p[i].pn;
index = i;

```
}
}

if (index != -1) {
    p[index].ct = time - p[index].at;
    time = p[index].bt;
    p[index].ct = time;
    p[index].tat = p[index].ct + p[index].at;
    p[index].wt = p[index].tat - p[index].bt;
    avg_tat += p[index].tat;
    avg_wt += p[index].wt;
    avg_rt += p[index].rt;
    p[index].completed = 1;
    completed++;
} else {
    time++;
}
}

printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].pn, p[i].ct, p[i].tat, p[i].wt, p[i].rt);
}

printf("Average Turnaround Time = %.2f", avg_tat/n);
printf("Average Waiting Time = %.2f", avg_wt/n);
printf("Average Response Time = %.2f", avg_rt/n);
}
```

return 0;

→ Output
Enter number of processes: 5

Process 1:
Arrival Time: 0
Burst Time: 3
Priority Time: 5

Process	2	3	4	5
2	3	4	6	
2	5	4	1	
3	2	4	1	

PID	AT	BT	PR	CT	TAT	WT	RT
1	0	3	5	3	3	0	0
2	2	2	3	11	9	7	7
3	3	5	2	8	5	0	0
4	4	4	4	15	11	7	7
5	6	1	1	9	3	2	2

Average Turnaround Time = 6.20
Average Waiting Time = 3.20
Average Response Time = 3.20

→ Gantt →

P ₁	P ₃	P ₅	P ₂	P ₄
0	3	8	9	15

Q4. WACP to simulate Priority Scheduling algorithm pre-emptive

```
#include <stdio.h>
#include <stdlib.h>
struct Process {
    int pid, at, bt, ct, pn, tat, wt, resp_t;
    bool started;
};

int main() {
    int n, time = 0, completed = 0;
    float total_tat = 0, total_wt = 0, total_resp_t = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i+1;
        printf("Process %d: ", i+1);
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Priority: ");
        scanf("%d", &p[i].pn);
        p[i].tat = p[i].bt;
        p[i].started = false;
    }

    while (completed < n) {
        int index = -1;
        int highest_pn = 9999;
        for (int i = 0; i < n; i++) {
            if (p[i].at <= time && !p[i].started && p[i].pn < highest_pn) {
                highest_pn = p[i].pn;
                index = i;
            }
        }

        if (index != -1) {
            p[index].ct = time - p[index].at;
            time = p[index].bt;
            p[index].ct = time;
            p[index].tat = p[index].ct + p[index].at;
            p[index].wt = p[index].tat - p[index].bt;
            p[index].resp_t = time - p[index].at;
            p[index].started = true;
            completed++;
        } else {
            time++;
        }
    }

    printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt, p[i].pn, p[i].ct, p[i].tat, p[i].wt, p[i].resp_t);
    }

    printf("Average Turnaround Time = %.2f", total_tat/n);
    printf("Average Waiting Time = %.2f", total_wt/n);
    printf("Average Response Time = %.2f", total_resp_t/n);
}
```


classmate
Date _____
Page _____

```

if (p[i].at < time && p[i].rt > 0 && p[i].pr <
    highest.pr) {
    highest.pr = p[i].pr;
    index = i;
}
if (index != -1) {
    if (!p[index].started) {
        p[index].resp_t = time - p[index].at;
        total_resp_t += p[index].resp_t;
        p[index].started = true;
    }
    p[index].ut--; time++;
    if (p[index].ut == 0) {
        completed++;
        p[index].ct = time;
        p[index].tat = p[index].ct - p[index].at;
        p[index].wt = p[index].tat - p[index].bt;
        p[index].at + p[index].tat = p[index].ct;
        total_wt += p[index].wt;
    }
    else {
        time++;
    }
}
printf("PID|AT|BT|PR|CT|TAT|WT|RT\n");
for (int i=0; i<n; i++) {
    printf("%d| %d| %d| %d| %d| %d| %d| %d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].pr, p[i].ct, p[i].tat, p[i].wt, p[i].rt);
}

```

classmate
Date _____
Page _____

```

p[i].resp_t);
}
printf("\nAverage Turnaround Time = %.2f", total_tat/n);
printf("\nAverage Waiting Time = %.2f", total_wt/n);
printf("\nAverage Response Time = %.2f", total_resp_t/n);
return 0;
}

```

→ Output

Enter number of processes: 5
 Process 1: 2: 3: 4: 5:
 Arrival Time 0 2 3 4 6
 Burst Time 3 2 5 4 1
 Priority: 5 3 2 4 1

PID	AT	BT	PR	CT	TAT	WT	RT
1	0	3	5	15	15	12	0
2	2	2	3	10	8	6	0
3	3	5	2	9	6	1	0
4	4	4	4	14	10	6	0
5	6	1	1	7	1	0	0

Average Turnaround Time = 8.00
 Average Waiting Time = 5.00
 Average Response Time = 1.20

→ Gantt →

P ₁	P ₂	P ₃	P ₃	P ₃	P ₃	P ₄	P ₁
0	2	3	6	7	9	10	15

classmate
Date _____
Page _____

1613126 #LAB-3

Q1. Round Robin Scheduling (with different quantum sizes)

```

#include <stdio.h>
#include <string.h>
#define MAX 10
#define QUEUE_SIZE (MAX*10)
typedef struct {
    int pid, at, bt, rt, ct, tat, wt;
} Process;
typedef struct {
    int data[QUEUE_SIZE];
    int front;
    int rear;
} Queue;
void initQueue(Queue *q) {
    q->front = 0;
    q->rear = 0;
}
int isEmpty(Queue *q) {
    return q->front == q->rear;
}
void enqueue(Queue *q, int val) {
    q->data[q->rear++] = val;
}
int dequeue(Queue *q) {
    return q->data[q->front++];
}
void addArrivals(Process proc[], int n, Queue *q,
    int inQueue[], int current_time) {

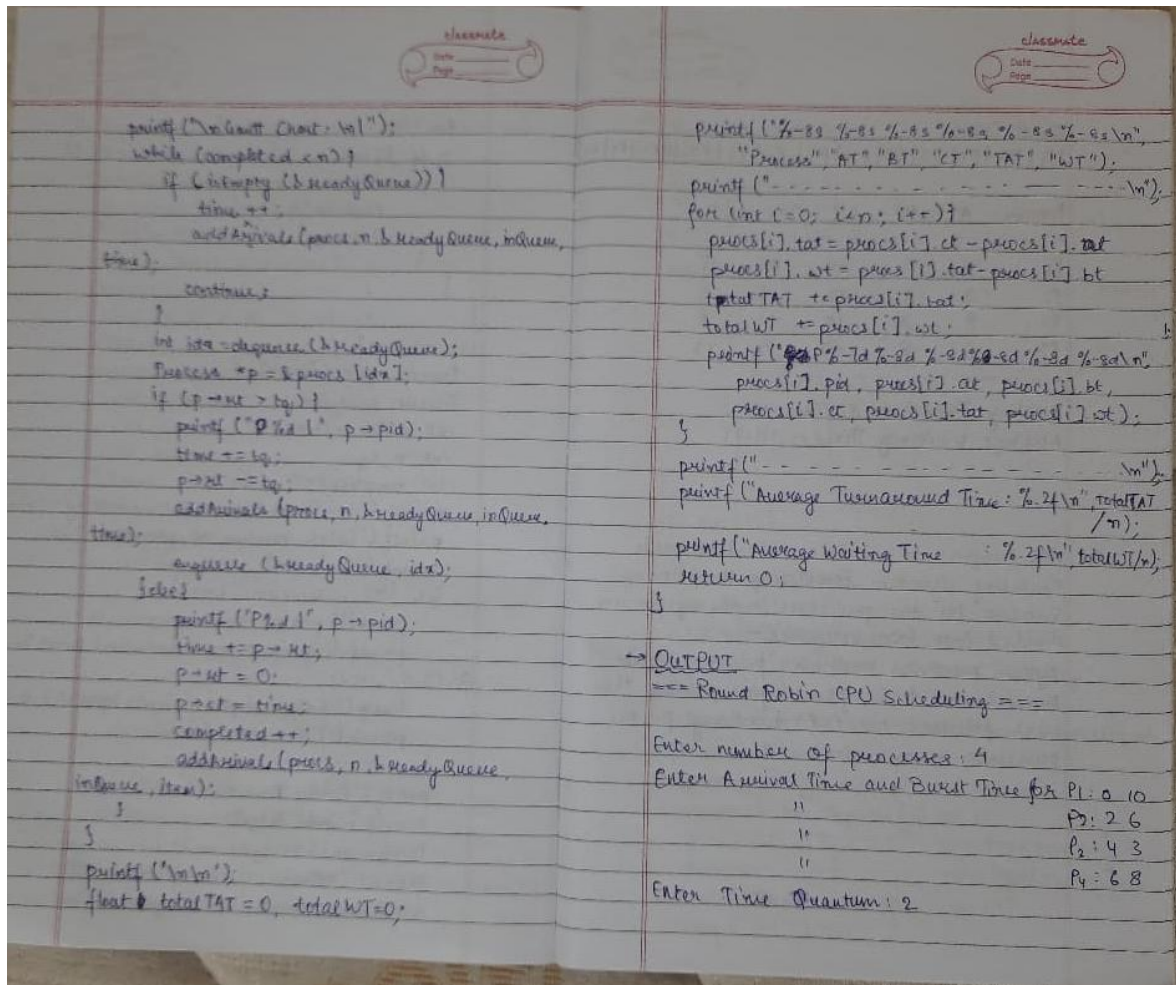
```

classmate
Date _____
Page _____

```

for (int i=0; i<n; i++) {
    if (!inQueue[i] && proc[i].rt > 0 &&
        proc[i].at <= current_time) {
        enqueue(q, i);
        inQueue[i] = 1;
    }
}
int main() {
    Process proc[MAX];
    Queue readyQueue;
    int inQueue[MAX];
    int n, tq;
    int completed=0, time=0;
    printf("== Round Robin CPU Scheduling ==\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (int i=0; i<n; i++) {
        proc[i].pid = i+1;
        printf("Enter Arrival Time and Burst Time\n");
        printf("for %d: ", i+1);
        scanf("%d %d", &proc[i].at, &proc[i].bt);
        proc[i].rt = proc[i].bt;
    }
    printf("Enter Time Quantum: ");
    scanf("%d", &tq);
    initQueue(&readyQueue);
    memset(inQueue, 0, sizeof(inQueue));
    addArrivals(proc, n, &readyQueue, inQueue, time);
}

```



Process	AT	BT	CT	TAT	WT
P1	0	10	25	25	15
P2	2	6	17	15	9
P3	4	3	15	11	8
P4	6	8	27	21	13

Average Turnaround Time : 18.00
 Average waiting Time : 11.25

- Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue

```
#include <stdio.h>
#define MAX 10
typedef struct {
    int pid, at, bt, type, start, ct, tat, wt, rt;
} Process;
void sortByAT(Process proc[], int idx[], int count) {
    for (int i = 0; i < count - 1; i++)
        for (int j = 0; j < count - i - 1; j++)
            if (proc[idx[j]].at > proc[idx[j + 1]].at) {
                int tmp = idx[j];
                idx[j] = idx[j + 1];
                idx[j + 1] = tmp;
            }
}
int main() {
    int n;
    Process proc[MAX];
    int sysQ[MAX], sysCount = 0;
    int usrQ[MAX], usrCount = 0;
    printf("=== Multilevel Queue Scheduling (System > User, FCFS) ===\n\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("\nProcess %d (PID AT BT): ", i + 1);
        scanf("%d %d %d", &proc[i].pid, &proc[i].at, &proc[i].bt);
        printf(" Type (0=System, 1=User): "); scanf("%d", &proc[i].type);
        if (proc[i].type == 0) sysQ[sysCount++] = i;
        else
            usrQ[usrCount++] = i;
    }
    sortByAT(proc, sysQ, sysCount);
    sortByAT(proc, usrQ, usrCount);
    int time = 0;
    printf("\nGantt Chart:\n|");
    for (int i = 0; i < sysCount; i++) {
        int p = sysQ[i];
        if (time < proc[p].at) time = proc[p].at;
        proc[p].start = time;
        proc[p].rt = proc[p].start - proc[p].at;
        printf(" P%d(S) |", proc[p].pid);
        time += proc[p].bt;
        proc[p].ct = time;
        proc[p].tat = proc[p].ct - proc[p].at;
        proc[p].wt = proc[p].tat - proc[p].bt;
    }
    for (int i = 0; i < usrCount; i++) {
        int p = usrQ[i];
        if (time < proc[p].at) time = proc[p].at;
        proc[p].start = time;
        proc[p].rt = proc[p].start - proc[p].at;
        printf(" P%d(U) |", proc[p].pid);
        time += proc[p].bt;
    }
}
```

```

    proc[p].ct  = time;
    proc[p].tat = proc[p].ct - proc[p].at;
    proc[p].wt  = proc[p].tat - proc[p].bt;
}
printf("\n\n");
float totalTAT = 0, totalWT = 0, totalRT = 0;
printf("%-6s %-6s %-6s %-8s %-6s %-6s %-6s %-6s\n",
        "PID", "AT", "BT", "Type", "CT", "TAT", "WT", "RT");
printf("-----\n");
for (int i = 0; i < sysCount; i++) {
    int p = sysQ[i];
    printf("%-6d %-6d %-6d %-8s %-6d %-6d %-6d %-6d\n",
            proc[p].pid, proc[p].at, proc[p].bt,
            "System", proc[p].ct, proc[p].tat, proc[p].wt, proc[p].rt);
    totalTAT += proc[p].tat;
    totalWT  += proc[p].wt;
    totalRT  += proc[p].rt;
}
for (int i = 0; i < usrCount; i++) {
    int p = usrQ[i];
    printf("%-6d %-6d %-6d %-8s %-6d %-6d %-6d %-6d\n",
            proc[p].pid, proc[p].at, proc[p].bt,
            "User", proc[p].ct, proc[p].tat, proc[p].wt, proc[p].rt);
    totalTAT += proc[p].tat;
    totalWT  += proc[p].wt;
    totalRT  += proc[p].rt;
}
printf("-----\n");
printf("Average Turnaround Time : %.2f\n", totalTAT / n);
printf("Average Waiting Time   : %.2f\n", totalWT / n);
printf("Average Response Time  : %.2f\n", totalRT / n);
return 0;
}

```

```

=== Multilevel Queue Scheduling (System > User, FCFS) ===

Enter number of processes: 4

Process 1 (PID AT BT): 1 0 4
Type (0=System, 1=User): 0

Process 2 (PID AT BT): 2 2 3
Type (0=System, 1=User): 0

Process 3 (PID AT BT): 3 1 5
Type (0=System, 1=User): 1

Process 4 (PID AT BT): 4 2 3
Type (0=System, 1=User): 1

Gantt Chart:
| P1(S) | P2(S) | P3(U) | P4(U) |

PID  AT  BT  Type  CT  TAT  WT  RT
-----
1    0    4   System  4    4    0    0
2    2    3   System  7    5    2    2
3    1    5   User    12   11    6    6
4    2    3   User    15   13   10   10
-----

Average Turnaround Time : 8.25
Average Waiting Time    : 4.50
Average Response Time   : 4.50

```



```

#include <stdio.h>
#define MAX 10
typedef struct {
    int pid, at, bt, type, start, ct, tat, wt, rt;
} Process;

void sortByAT (Process pproc[], int idx1, int count) {
    for (int i=0; i<count-1; i++)
        for (int j=i+1; j<count; j++)
            if (pproc[idx1].at > pproc[j].at) {
                int tmp = pproc[i];
                pproc[i] = pproc[j];
                pproc[j] = tmp;
            }
}

int main() {
    int n;
    Process pproc[MAX];
    int sysQ[MAX], sysCount=0;
    int userQ[MAX], userCount=0;
    printf("== Multilevel Queue Scheduling\n");
    printf("System > User, FCFS ==\n");
    scanf("%d", &n);
    for (int i=0; i<n; i++) {
        printf("Type (0=System, 1=User): ");
        scanf("%d", &pproc[i].type);
        if (pproc[i].type == 0) sysQ[sysCount++] = i;
        else userQ[userCount++] = i;
    }
    printf("Enter number of processes: ");
    scanf("%d", &n);
}

```

```

for (int i=0; i<n; i++)
    printf("PID AT BT: ", i+1);
    scanf("%d %d", &pproc[i].pid, &pproc[i].at, &pproc[i].bt);
    printf("Type (0=System, 1=User): ");
    scanf("%d", &pproc[i].type);
    if (pproc[i].type == 0) sysQ[sysCount++] = i;
    else userQ[userCount++] = i;

sortByAT(pproc, sysQ, sysCount);
sortByAT(pproc, userQ, userCount);
int time = 0;
printf("\n Gantt Chart: ");
for (int i=0; i<sysCount; i++) {
    int p = sysQ[i];
    if (time < pproc[p].at) time = pproc[p].at;
    pproc[p].start = time;
    pproc[p].wt = pproc[p].start - pproc[p].at;
    printf("P%d(s) |", pproc[p].pid);
    time += pproc[p].bt;
    pproc[p].ct = time;
    pproc[p].tat = pproc[p].ct - pproc[p].at;
    pproc[p].rt = pproc[p].tat - pproc[p].wt;
}
for (int i=0; i<userCount; i++) {
    int p = userQ[i];
    if (time < pproc[p].at) time = pproc[p].at;
    pproc[p].start = time;
    pproc[p].rt = pproc[p].start - pproc[p].at;
}

```

```

printf("P %d(s) |", pproc[p].pid);
time += pproc[p].bt;
pproc[p].ct = time;
pproc[p].tat = pproc[p].ct - pproc[p].at;
pproc[p].rt = pproc[p].tat - pproc[p].wt;
}
printf("\n\n");
float totalTAT=0, totalWT=0, totalRT=0;
printf("Total TAT: %f, Total WT: %f, Total RT: %f\n", totalTAT, totalWT, totalRT);
for (int i=0; i<sysCount; i++) {
    int p = sysQ[i];
    printf("P %d(s) |", pproc[p].pid);
    pproc[p].start = pproc[p].at;
    pproc[p].wt = pproc[p].start - pproc[p].at;
    totalTAT += pproc[p].tat;
    totalWT += pproc[p].wt;
    totalRT += pproc[p].rt;
}
for (int i=0; i<userCount; i++) {
    int p = userQ[i];
    printf("P %d(s) |", pproc[p].pid);
    pproc[p].start = pproc[p].at;
    pproc[p].wt = pproc[p].start - pproc[p].at;
    totalTAT += pproc[p].tat;
    totalWT += pproc[p].wt;
    totalRT += pproc[p].rt;
}

```

```

printf("==\n");
printf("Average Turnaround Time : %.2f\n", totalTAT/n);
printf("Average Waiting Time : %.2f\n", totalWT/n);
printf("Average Response Time : %.2f\n", totalRT/n);
return 0;
}

```

Output

== Multilevel Queue Scheduling (System > User, FCFS) ==

Enter number of processes: 4

Process 1 (PID AT BT): 1 0 4

Type (0=System, 1=User): 0

" 2 " : 2 2 3 / 0

" 3 " : 3 1 5 / 0

" 4 " : 4 2 3 / 1

Gantt Chart:

| P1(S) | P2(S) | P3(U) | P4(U) |

PID	AT	BT	Type	CT	TAT	WT	RT
1	0	4	System	4	4	0	0
2	2	3	System	7	5	2	2
3	1	5	User	12	11	6	6
4	2	3	User	15	13	10	10

Average Turnaround Time : 8.25

Average Waiting Time : 4.50

Average Response Time : 4.50

23/03/26 LAB-4

Sample →

P₁
P₂
P₃
P₄
P₁
P₂
P₃
P₄

Rate

Code →

#inc

#in

#in

#de

type

i

int

int

int

int

int

int

int

int

int

int

int

int

int

int

3. Write a C program to simulate Real-Time CPU Scheduling algorithms
 - a) Rate- Monotonic

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_TASKS 10
#define MAX_TIME 100
typedef struct {
    int id, C, P, next_arrival, remaining_time, deadline_miss;
} Task;
int compare_period(const void *a, const void *b) {
    return ((Task *)a)->P - ((Task *)b)->P;
}
void print_gantt(int gantt[], int max_time, int n_tasks, Task tasks[]) {
    printf("\n--- Gantt Chart ---\nTime: ");
    for (int t = 0; t < max_time; t++) printf("%3d", t);
    printf("\n");
    for (int i = 0; i < n_tasks; i++) {
        printf("T%d : ", tasks[i].id);
        for (int t = 0; t < max_time; t++) printf(gantt[t] == tasks[i].id ? " #" : " .");
        printf("\n");
    }
    printf("CPU : ");
    for (int t = 0; t < max_time; t++) {
        if (gantt[t] == -1) printf(" _"); else printf(" T%d", gantt[t]);
    }
    printf("\n");
}
int main() {
    int n;
    Task tasks[MAX_TASKS];
    printf("=== Rate Monotonic Scheduling Simulator ===\n\nEnter number of tasks (max %d): ", MAX_TASKS);
    scanf("%d", &n);
    if (n <= 0 || n > MAX_TASKS) { printf("Invalid number of tasks.\n"); return 1; }
    for (int i = 0; i < n; i++) {
        tasks[i].id = i + 1;
        printf("\nTask T%d:\n Execution time (C%d): ", i + 1, i + 1);
        scanf("%d", &tasks[i].C);
        printf(" Period (P%d): ", i + 1);
        scanf("%d", &tasks[i].P);
        if (tasks[i].C <= 0 || tasks[i].P <= 0 || tasks[i].C > tasks[i].P) {
            printf("Invalid: ensure C > 0, P > 0, and C <= P.\n"); return 1;
        }
        tasks[i].next_arrival = tasks[i].remaining_time = tasks[i].deadline_miss = 0;
    }
    qsort(tasks, n, sizeof(Task), compare_period);
    printf("\n--- Priority Assignment (lower index = higher priority) ---\n%-8s %-8s %-8s %-12s\n", "Task", "C", "P",
"Priority");
    for (int i = 0; i < n; i++) printf("T%-7d %-8d %-8d %-12d\n", tasks[i].id, tasks[i].C, tasks[i].P, i + 1);
    double util = 0.0;
    for (int i = 0; i < n; i++) util += (double)tasks[i].C / tasks[i].P;
    printf("\nUtilization: %.4f\n", util);
    int lcm = 1;
    for (int i = 0; i < n; i++) {
        int a = lcm, b = tasks[i].P;
```



```

while (b) { int t = b; b = a % b; a = t; }
lcm = lcm / a * tasks[i].P;
if (lcm > MAX_TIME) { lcm = MAX_TIME; break; }
}
int max_time = lcm, gantt[MAX_TIME];
memset(gantt, -1, sizeof(gantt));
for (int i = 0; i < n; i++) tasks[i].next_arrival = tasks[i].remaining_time = 0;
printf("\nSimulation length: %d time units\n\n--- Scheduling Log ---\n%-6s %-12s %-10s\n", max_time, "Time",
"Running", "Event");
for (int time = 0; time < max_time; time++) {
    for (int i = 0; i < n; i++) {
        if (time == tasks[i].next_arrival) {
            if (tasks[i].remaining_time > 0) {
                printf("t=%-4d %-12s DEADLINE MISS for T%d!\n", time, "---", tasks[i].id);
                tasks[i].deadline_miss++;
            }
            tasks[i].remaining_time = tasks[i].C;
            tasks[i].next_arrival += tasks[i].P;
        }
    }
}
int current = -1;
for (int i = 0; i < n; i++) { if (tasks[i].remaining_time > 0) { current = i; break; } }
if (current != -1) {
    gantt[time] = tasks[current].id;
    tasks[current].remaining_time--;
    printf("t=%-4d T%-11d (remaining: %d)\n", time, tasks[current].id, tasks[current].remaining_time);
} else {
    gantt[time] = -1;
    printf("t=%-4d %-12s\n", time, "IDLE");
}
}
print_gantt(gantt, max_time, n, tasks);
printf("\n--- Summary ---\n");
int any_miss = 0;
for (int i = 0; i < n; i++) {
    printf("T%d: %d deadline miss(es)\n", tasks[i].id, tasks[i].deadline_miss);
    if (tasks[i].deadline_miss > 0) any_miss = 1;
}
printf(any_miss ? "Deadline misses detected.\n" : "All deadlines met. Schedule is feasible.\n");
return 0;
}

```

```

--- Gantt Chart ---
Time:  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19
T4  :  #  .  .  .  #  .  .  .  #  .  .  .  #  .  .  .  #  .  .  .
T2  :  .  #  #  .  .  #  #  .  .  .  #  #  .  .  .  #  .  #  .  .
T3  :  .  .  .  #  .  .  .  #  .  .  .  .  #  #  .  .  .  .  .
T1  :  .  .  .  .  .  .  .  .  .  #  .  .  .  .  .  .  .  #  #
CPU  :  T4 T2 T2 T3 T4 T2 T2 T3 T4 T1 T2 T2 T4 T3 T3 T2 T4 T2 T1 T1

--- Summary ---
T4: 0 deadline miss(es)
T2: 0 deadline miss(es)
T3: 0 deadline miss(es)
T1: 0 deadline miss(es)
All deadlines met. Schedule is feasible.

```

```

=== Rate Monotonic Scheduling Simulator ===

Enter number of tasks (max 10): 4

Task T1:
Execution time (C1): 3
Period (P1): 20

Task T2:
Execution time (C2): 2
Period (P2): 5

Task T3:
Execution time (C3): 2
Period (P3): 10

Task T4:
Execution time (C4): 1
Period (P4): 4

--- Priority Assignment (lower index = higher priority) ---
Task  C      P      Priority
T4    1      4      1
T2    2      5      2
T3    2     10      3
T1    3     20      4

Utilization: 1.0000

Simulation length: 20 time units

--- Scheduling Log ---
Time  Running  Event
t=0   T4       (remaining: 0)
t=1   T2       (remaining: 1)
t=2   T2       (remaining: 0)
t=3   T3       (remaining: 1)
t=4   T4       (remaining: 0)
t=5   T2       (remaining: 1)
t=6   T2       (remaining: 0)
t=7   T3       (remaining: 0)
t=8   T4       (remaining: 0)
t=9   T1       (remaining: 2)
t=10  T2       (remaining: 1)
t=11  T2       (remaining: 0)
t=12  T4       (remaining: 0)
t=13  T3       (remaining: 1)
t=14  T3       (remaining: 0)
t=15  T2       (remaining: 1)
t=16  T4       (remaining: 0)
t=17  T2       (remaining: 0)
t=18  T1       (remaining: 1)
t=19  T1       (remaining: 0)

```

b)Proportional Share Scheduling

```

#include <stdio.h>
struct Process {
    int pid,is_completed;
    float arrival_time,burst_time,weight,remaining_time,completion_time,turnaround_time,waiting_time;
};
struct Gantt {
    int pid;
    float start;
    float end;
};
int main() {
    int n;
    float time_quantum;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the Base Time Quantum: ");
    scanf("%f", &time_quantum);
    struct Process p[n];
    struct Gantt chart[100];

```

```

int chart_count = 0;
float total_weight = 0.0;
float total_tat = 0.0, total_wt = 0.0;
for (int i = 0; i < n; i++) {
    p[i].pid = i + 1;
    printf("\nProcess %d\n", p[i].pid);
    printf("Enter Arrival Time: ");
    scanf("%f", &p[i].arrival_time);
    printf("Enter Burst Time: ");
    scanf("%f", &p[i].burst_time);
    printf("Enter Weight: ");
    scanf("%f", &p[i].weight);
    p[i].remaining_time = p[i].burst_time;
    p[i].is_completed = 0;
    total_weight += p[i].weight;
}
float current_time = 0.0;
int completed_processes = 0;
while (completed_processes < n) {
    int executed_any = 0;
    for (int i = 0; i < n; i++) {
        if (p[i].arrival_time <= current_time && p[i].is_completed == 0) {
            if (total_weight <= 0) break;
            float time_slice = (p[i].weight / total_weight) * time_quantum;
            if (p[i].remaining_time < time_slice) {
                time_slice = p[i].remaining_time;
            }
            chart[chart_count].pid = p[i].pid;
            chart[chart_count].start = current_time;
            current_time += time_slice;
            p[i].remaining_time -= time_slice;
            chart[chart_count].end = current_time;
            chart_count++;
            executed_any = 1;
            if (p[i].remaining_time <= 0.0001) {
                p[i].remaining_time = 0;
                p[i].is_completed = 1;
                completed_processes++;
                p[i].completion_time = current_time;
                p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
                p[i].waiting_time = p[i].turnaround_time - p[i].burst_time;
                total_tat += p[i].turnaround_time;
                total_wt += p[i].waiting_time;
                total_weight -= p[i].weight;
            }
        }
    }
    if (executed_any == 0) {
        chart[chart_count].pid = 0;
        chart[chart_count].start = current_time;
        current_time += 1.0;
        chart[chart_count].end = current_time;
        chart_count++;
    }
}
printf("\n-----\n");

```

```

printf("PID\tArrival\tBurst\tWeight\tCompletion\tTurnaround\tWaiting\n");
printf("-----\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%.2f\t%.2f\t%.2f\t%.2f\t%.2f\t%.2f\n",
        p[i].pid, p[i].arrival_time, p[i].burst_time, p[i].weight,
        p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
}
printf("-----\n");
printf("\n--- GANTT CHART ---\n");
for (int i = 0; i < chart_count; i++) printf("-----");
printf("\n");
for (int i = 0; i < chart_count; i++) {
    if (chart[i].pid == 0) printf(" IDLE |");
    else printf(" P%d |", chart[i].pid);
}
printf("\n");
for (int i = 0; i < chart_count; i++) printf("-----");
printf("\n0.00");
for (int i = 0; i < chart_count; i++) {
    printf(" %.2f", chart[i].end);
}
printf("\n\nAverage Turn Around Time: %.2f", total_tat / n);
printf("\nAverage Waiting Time: %.2f\n", total_wt / n);
return 0;
}

```

```

Enter the number of processes: 3
Enter the Base Time Quantum: 6

```

```

Process 1
Enter Arrival Time: 0
Enter Burst Time: 2
Enter Weight: 1

```

```

Process 2
Enter Arrival Time: 1
Enter Burst Time: 3
Enter Weight: 2

```

```

Process 3
Enter Arrival Time: 2
Enter Burst Time: 4
Enter Weight: 3

```

PID	Arrival	Burst	Weight	Completion	Turnaround	Waiting
1	0.00	2.00	1.00	7.00	7.00	5.00
2	1.00	3.00	2.00	8.00	7.00	4.00
3	2.00	4.00	3.00	9.00	7.00	3.00

```

--- GANTT CHART ---

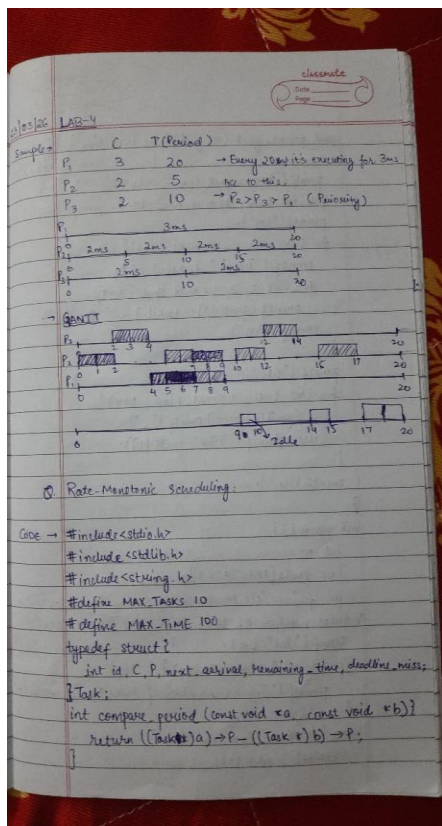
```

P1	P2	P3	P1	P2	P3	
0.00	1.00	3.00	6.00	7.00	8.00	9.00

```

Average Turn Around Time: 7.00
Average Waiting Time: 4.00

```



classmate

```

void print_gantt (int gantt[], int max_time, int n, Task task[]) {
    printf("m--- Gantt Chart ---\n");
    for (int t=0; t<max_time; t++) printf("%3d", t);
    printf("\n");
    for (int i=0; i<n; i++) {
        printf("T%d", task[i].id);
        for (int t=0; t<max_time; t++) {
            printf("gantt[%d] = %d", task[i].id, gantt[t]);
            printf("\n");
        }
    }
    printf("CP\n");
    for (int t=0; t<max_time; t++) {
        if (gantt[t] == -1) printf("-");
        else printf("T%d", gantt[t]);
    }
    printf("\n");
}

int main () {
    int n;
    Task tasks[MAX_TASKS];
    printf("== Rate Monotonic Scheduling Simulation ==\n");
    // Enter number of tasks (max %d): ", MAX_TASKS);
    scanf ("%d", &n);
    if (n <= 0 || n > MAX_TASKS) {
        printf("Invalid number of tasks.\n"); return 1;
    }
    for (int i=0; i<n; i++) {
        tasks[i].id = i+1;
    }
}
  
```

classmate

```

        printf("Task To %d\n Execution Time (C):",
        i+1, i+1);
        scanf ("%d", &tasks[i].C);
        printf("Period (T):", i+1);
        scanf ("%d", &tasks[i].P);
        if (tasks[i].C <= 0 || tasks[i].P <= 0 || tasks[i].C >
            tasks[i].P) {
            printf("Invalid: ensure C > 0, P > 0, and C <= P\n");
            return 1;
        }
        tasks[i].next_arrival = tasks[i].remaining_time;
    }

    sort(tasks, n, sizeof (Task), compare_period);
    printf("m--- Priority Assignment (lower index: higher
    priority) ---\n %3d %3d %3d %3d %3d %3d %3d %3d %3d %3d", "Task", "C", "P",
    "Priority");
    for (int i=0; i<n; i++) {
        printf("T%d %3d %3d %3d %3d", task[i].id,
        tasks[i].C, tasks[i].P, i+1);
        double util = 0.0;
        for (int i=0; i<n; i++) {
            util += (double)tasks[i].C / tasks[i].P;
            printf("Utilization: %.4f\n", util);
            int lcm = 1;
            for (int i=0; i<n; i++) {
                int a = lcm, b = tasks[i].P;
                while (b % a != 0) {
                    a = lcm * b; b = a % b; a = b;
                }
                lcm = lcm * a / tasks[i].P;
                if (lcm > MAX_TIME) { lcm = MAX_TIME; break; }
            }
        }
    }
}
  
```

classmate

```

    int max_time = lcm, gantt[MAX_TIME];
    memset (gantt, -1, sizeof (gantt));
    for (int time=0; time<max_time; time++) {
        for (int i=0; i<n; i++) {
            if (time == tasks[i].next_arrival) {
                if (tasks[i].remaining_time > 0) {
                    printf("t = %3d %3d %3d %3d %3d %3d %3d %3d %3d %3d", "Task", "id", "C", "P", "T", "next_arrival", "remaining_time", "deadline_miss");
                    tasks[i].remaining_time--;
                    tasks[i].next_arrival += tasks[i].P;
                }
            }
        }
        int current = -1;
        for (int i=0; i<n; i++) {
            if (tasks[i].remaining_time > 0) {
                current = i; break;
            }
        }
        if (current == -1) {
            gantt[time] = tasks[current].id;
            tasks[current].remaining_time--;
            printf("t = %3d %3d %3d %3d %3d %3d %3d %3d %3d %3d", "Task", "id", "C", "P", "T", "next_arrival", "remaining_time", "deadline_miss");
            gantt[time] = -1;
            printf("t = %3d %3d %3d %3d %3d %3d %3d %3d %3d %3d", "Task", "id", "C", "P", "T", "next_arrival", "remaining_time", "deadline_miss");
        }
    }
    print_gantt (gantt, max_time, n, tasks);
    printf("m--- Summary ---\n");
}
  
```

classmate

```

    int any_miss = 0;
    for (int i=0; i<n; i++) {
        printf("T%d: %3d %3d %3d %3d %3d %3d %3d %3d %3d %3d", "Task", "id", "C", "P", "T", "next_arrival", "remaining_time", "deadline_miss");
        if (tasks[i].deadline_miss > 0) any_miss = 1;
    }
    printf("any_miss? Deadline missed detected.\n");
    if (any_miss) {
        printf("All deadlines met. Schedule is feasible.\n");
        return 0;
    }
}

Output
== Rate Monotonic Scheduling Simulation ==
Enter number of tasks (max 10): 4
Task T1:
    Execution Time (C1): 3 Period (P1): 20
Task T2:
    Execution Time (C2): 2 Period (P2): 5
Task T3:
    Execution Time (C3): 2 Period (P3): 10
Task T4:
    Execution Time (C4): 1 Period (P4): 4

--- Priority Assignment (lower index: higher priority) ---
Task C P Priority
T4 1 4 1
T2 2 5 2
T3 2 10 3
T1 3 20 4

--- Summary ---
T4: 0 deadline miss(es)
T3: 0 deadline miss(es)
T2: 0 deadline miss(es)
T1: 0 deadline miss(es)
All deadlines met. Schedule is feasible
  
```

classmate

Utilization: 1.0000
Simulation length: 20 time units

--- Scheduling log ---

Time	Running	Event
0	T4	Task T4 arrives
1	T4	Task T4 completes
2	T2	Task T2 arrives
3	T2	Task T2 completes
4	T4	Task T4 arrives
5	T4	Task T4 completes
6	T3	Task T3 arrives
7	T3	Task T3 completes
8	T4	Task T4 arrives
9	T4	Task T4 completes
10	T2	Task T2 arrives
11	T2	Task T2 completes
12	T3	Task T3 arrives
13	T3	Task T3 completes
14	T4	Task T4 arrives
15	T4	Task T4 completes
16	T2	Task T2 arrives
17	T2	Task T2 completes
18	T3	Task T3 arrives
19	T3	Task T3 completes
20	T4	Task T4 arrives
21	T4	Task T4 completes

--- Summary ---

T4: 0 deadline miss(es)
T3: 0 deadline miss(es)
T2: 0 deadline miss(es)
T1: 0 deadline miss(es)
All deadlines met. Schedule is feasible

4. Write a C program to simulate:

a) Producer-Consumer problem using semaphores.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 5
#define NUM_PRODUCERS 2
#define NUM_CONSUMERS 2
#define NUM_ITEMS 10
int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
sem_t empty;
sem_t full;
pthread_mutex_t mutex;
void* producer(void* param) {
    int producer_id = *((int*)param);
    for (int i = 0; i < NUM_ITEMS; i++) {
        int item = rand() % 100;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Producer %d: Inserted item %d at index %d\n", producer_id, item, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}
void* consumer(void* param) {
    int consumer_id = *((int*)param);
    for (int i = 0; i < NUM_ITEMS; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("Consumer %d: Removed item %d from index %d\n", consumer_id, item, out);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(2);
    }
    return NULL;
}
int main() {
    pthread_t producers[NUM_PRODUCERS];
    pthread_t consumers[NUM_CONSUMERS];
    int prod_ids[NUM_PRODUCERS];
    int cons_ids[NUM_CONSUMERS];
    pthread_mutex_init(&mutex, NULL);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
```



```

for (int i = 0; i < NUM_PRODUCERS; i++) {
    prod_ids[i] = i + 1;
    pthread_create(&producers[i], NULL, producer, &prod_ids[i]);
}
for (int i = 0; i < NUM_CONSUMERS; i++) {
    cons_ids[i] = i + 1;
    pthread_create(&consumers[i], NULL, consumer, &cons_ids[i]);
}
for (int i = 0; i < NUM_PRODUCERS; i++) {
    pthread_join(producers[i], NULL);
}
for (int i = 0; i < NUM_CONSUMERS; i++) {
    pthread_join(consumers[i], NULL);
}
pthread_mutex_destroy(&mutex);
sem_destroy(&empty);
sem_destroy(&full);

printf("All threads have finished execution.\n");
return 0;
}

```

```

Producer 2: Inserted item 0 at index 2
Consumer 1: Removed item 34 from index 4
Consumer 2: Removed item 34 from index 0
Producer 2: Inserted item 69 at index 3
Producer 1: Inserted item 69 at index 4
Producer 1: Inserted item 24 at index 0
Consumer 2: Removed item 0 from index 1
Consumer 1: Removed item 0 from index 2
Producer 2: Inserted item 24 at index 1
Producer 1: Inserted item 78 at index 2
Consumer 2: Removed item 69 from index 3
Consumer 1: Removed item 69 from index 4
Producer 2: Inserted item 78 at index 3
Producer 1: Inserted item 58 at index 4
Consumer 1: Removed item 24 from index 0
Consumer 2: Removed item 24 from index 1
Producer 1: Inserted item 62 at index 0
Producer 2: Inserted item 58 at index 1
Consumer 1: Removed item 78 from index 2
Consumer 2: Removed item 78 from index 3
Producer 2: Inserted item 62 at index 2
Producer 1: Inserted item 64 at index 3
Consumer 1: Removed item 58 from index 4
Consumer 2: Removed item 62 from index 0
Producer 2: Inserted item 64 at index 4
Consumer 1: Removed item 58 from index 1
Consumer 2: Removed item 62 from index 2
Consumer 2: Removed item 64 from index 3
Consumer 1: Removed item 64 from index 4
All threads have finished execution.

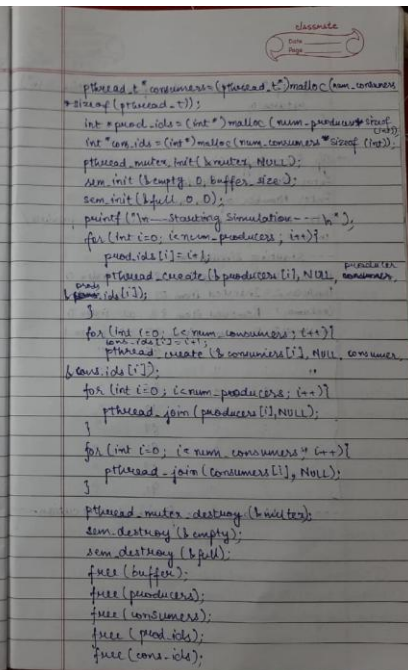
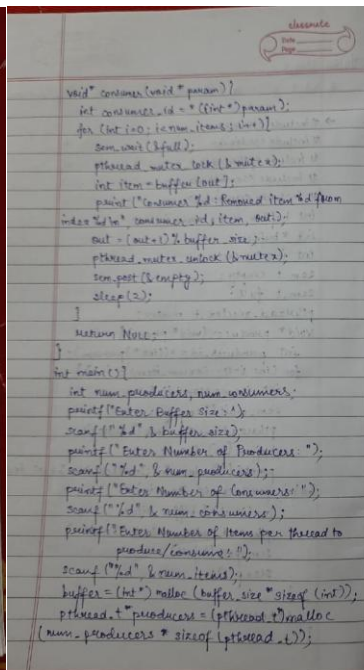
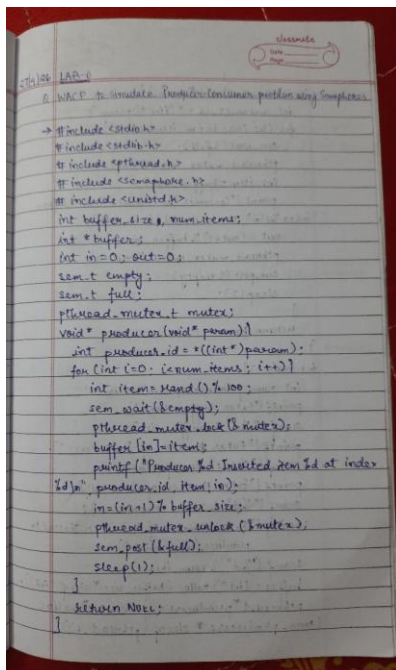
```

b) Dining-Philosopher's Problem

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#define NUM_PHILOSOPHERS 5
pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
void* philosopher(void* arg) {
    int id = *(int*)arg;
    int left_chopstick = id;
    int right_chopstick = (id + 1) % NUM_PHILOSOPHERS;
    for (int i = 0; i < 2; i++) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(rand() % 2 + 1);
        if (id % 2 == 0) {
            pthread_mutex_lock(&chopsticks[left_chopstick]);
            pthread_mutex_lock(&chopsticks[right_chopstick]);
        } else {
            pthread_mutex_lock(&chopsticks[right_chopstick]);
            pthread_mutex_lock(&chopsticks[left_chopstick]);
        }
        printf("Philosopher %d picked up chopsticks %d and %d and is EATING.\n", id, left_chopstick,
right_chopstick);
        sleep(rand() % 2 + 1);
        pthread_mutex_unlock(&chopsticks[left_chopstick]);
        pthread_mutex_unlock(&chopsticks[right_chopstick]);
        printf("Philosopher %d finished eating and put down chopsticks.\n", id);
    }
    return NULL;
}
int main() {
    pthread_t philosophers[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];
    srand(time(NULL));
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        if (pthread_mutex_init(&chopsticks[i], NULL) != 0) {
            printf("Mutex initialization failed.\n");
            return 1;
        }
    }
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        if (pthread_create(&philosophers[i], NULL, philosopher, &ids[i]) != 0) {
            printf("Thread creation failed.\n");
            return 1;
        }
    }
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(philosophers[i], NULL);
    }
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&chopsticks[i]);
    }
    printf("All philosophers have finished dining.\n");
    return 0;
}
```

}

```
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 1 picked up chopsticks 1 and 2 and is EATING.
Philosopher 3 picked up chopsticks 3 and 4 and is EATING.
Philosopher 3 finished eating and put down chopsticks.
Philosopher 3 is thinking.
Philosopher 0 picked up chopsticks 0 and 1 and is EATING.
Philosopher 1 finished eating and put down chopsticks.
Philosopher 1 is thinking.
Philosopher 2 picked up chopsticks 2 and 3 and is EATING.
Philosopher 0 finished eating and put down chopsticks.
Philosopher 0 is thinking.
Philosopher 4 picked up chopsticks 4 and 0 and is EATING.
Philosopher 2 finished eating and put down chopsticks.
Philosopher 2 is thinking.
Philosopher 1 picked up chopsticks 1 and 2 and is EATING.
Philosopher 4 finished eating and put down chopsticks.
Philosopher 4 is thinking.
Philosopher 3 picked up chopsticks 3 and 4 and is EATING.
Philosopher 3 finished eating and put down chopsticks.
Philosopher 0 picked up chopsticks 0 and 1 and is EATING.
Philosopher 1 finished eating and put down chopsticks.
Philosopher 2 picked up chopsticks 2 and 3 and is EATING.
Philosopher 2 finished eating and put down chopsticks.
Philosopher 0 finished eating and put down chopsticks.
Philosopher 4 picked up chopsticks 4 and 0 and is EATING.
Philosopher 4 finished eating and put down chopsticks.
All philosophers have finished dining.
```



classmate
Date _____
Page _____

```

printf("-- All threads have finished execution--\n");
return 0;
}

Op: Enter Buffer Size: 5
Enter Number of Producers: 2
Enter Number of Consumers: 2
Enter Number of Items per thread to produce/
consume: 3

--- Starting Simulation ---
Producer 1: Inserted item 83 at index 0
Producer 2: Inserted item 87 at index 1
Consumer 1: Removed item 83 at index 0
Consumer 2: Removed item 27 at index 1

" : " 45
" : " 12
" : " 45
" : " 12
" : " 67
" : " 91
" : " 67
" : " 91

--- All threads have finished execution ---

```

classmate
Date _____
Page _____

4/5/26 LAB-7

Q. WAP to simulate Dining Philosopher Problem

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#define NUM_PHILOSOPHERS 5

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
void* philosopher(void* arg) {
    int id = *(int*) arg;
    int left_chopstick = id;
    int right_chopstick = (id+1)%NUM_PHILOSOPHERS;
    for (int i=0; i<2; i++) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(rand() % 2 + 1);
        if (i % 2 == 0) {
            pthread_mutex_lock(&chopsticks[left_chopstick]);
            pthread_mutex_lock(&chopsticks[right_chopstick]);
        } else {
            pthread_mutex_lock(&chopsticks[right_chopstick]);
            pthread_mutex_lock(&chopsticks[left_chopstick]);
        }
        printf("Philosopher %d picked up chopsticks %d and %d\n", id, left_chopstick, right_chopstick);
        sleep(rand() % 2 + 1);
        pthread_mutex_unlock(&chopsticks[left_chopstick]);
        pthread_mutex_unlock(&chopsticks[right_chopstick]);
        printf("Philosopher %d finished eating and put down chopsticks.\n", id);
    }
}

```

classmate
Date _____
Page _____

```

return NULL;
}

int main() {
    pthread_t philosophers[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];
    srand(time(NULL));
    for (int i=0; i<NUM_PHILOSOPHERS; i++) {
        if (pthread_mutex_init(&chopsticks[i], NULL) != 0) {
            printf("Mutex initialization failed.\n");
            return 1;
        }
    }
    for (int i=0; i<NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        if (pthread_create(&philosophers[i], NULL, philosopher, &ids[i]) != 0) {
            printf("Thread creation failed.\n");
            return 1;
        }
    }
    for (int i=0; i<NUM_PHILOSOPHERS; i++) {
        pthread_join(philosophers[i], NULL);
    }
    for (int i=0; i<NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&chopsticks[i]);
    }
    printf("All philosophers have finished dining.\n");
    return 0;
}

```

classmate
Date _____
Page _____

Op: Philosopher 1 is thinking.

```

" 1
" 0
" 3
" 4
" 2

Philosopher 3 picked up chopsticks 3 and 4 and is EATING.
" 3 finished eating & put down chopsticks.
" 0 picked up chopsticks 0 and 1 and is EATING.
" 2 picked up 1
" 1 finished
" 1 thinking
" 3
" 2 finished
" 4 picked up
" 1 picked up
" 0 finished
" 0 is thinking
" 2
" 2 picked up
" 1 finished
" 2 finished
" 3 picked up
" 0 picked up
" 4 finished
" 4 is thinking
" 0 finished
" 3 finished
" 4 picked up
" 1 finished

All Philosophers have finished dining.

```

5. Write a C program to simulate:
Bankers' algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, m;
    printf("Enter the number of Processes: ");
    scanf("%d", &n);
    printf("Enter the number of Resource types: ");
    scanf("%d", &m);
    int alloc[n][m], max[n][m], avail[m], need[n][m];
    int f[n], ans[n], ind = 0;
    printf("\nEnter the Allocation Matrix:\n");
    for (int i = 0; i < n; i++) {
        printf("Process P%d: ", i);
        for (int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    }
    printf("\nEnter the Max Matrix:\n");
    for (int i = 0; i < n; i++) {
        printf("Process P%d: ", i);
        for (int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);
    }
    printf("\nEnter the Available Resources: ");
    for (int i = 0; i < m; i++)
        scanf("%d", &avail[i]);
    for (int k = 0; k < n; k++) f[k] = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    }
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            if (f[i] == 0) {
```

```

    int flag = 0;
    for (int j = 0; j < m; j++) {
        if (need[i][j] > avail[j]) {
            flag = 1;
            break;
        }
    }
    if (flag == 0) {
        ans[ind++] = i;
        for (int y = 0; y < m; y++)
            avail[y] += alloc[i][y];
        f[i] = 1;
    }
}

}

printf("\n--- Result ---\n");
bool safe = true;
for (int i = 0; i < n; i++) {
    if (f[i] == 0) {
        safe = false;
        printf("The system is in an UNSAFE state (Deadlock possible).\n");
        break;
    }
}

if (safe) {
    printf("The system is in a SAFE state.\nSafe Sequence: ");
    for (int i = 0; i < n - 1; i++)
        printf("P%d -> ", ans[i]);
    printf("P%d\n", ans[n - 1]);
}

return 0;
}

```


Enter the number of Processes: 5
Enter the number of Resource types: 3

Enter the Allocation Matrix:

Process P0: 0 1 0

Process P1: 2 0 0

Process P2: 3 0 2

Process P3: 2 1 1

Process P4: 0 0 2

Enter the Max Matrix:

Process P0: 7 5 3

Process P1: 3 2 2

Process P2: 9 0 2

Process P3: 2 2 2

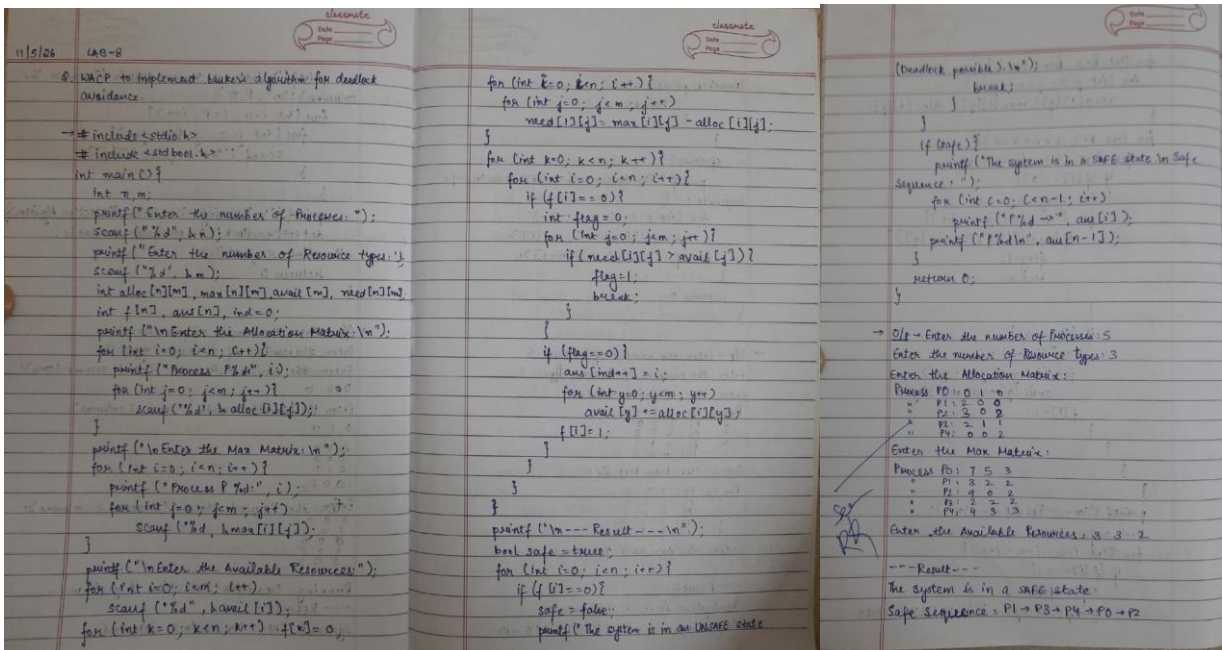
Process P4: 4 3 3

Enter the Available Resources: 3 3 2

--- Result ---

The system is in a SAFE state.

Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2



6. Write a C program to simulate the following contiguous memory allocation techniques.
- a) Worst-Fit
 - b) Best-Fit
 - c) First-Fit

```
#include <stdio.h>

void firstFit(int blockSize[], int blocks, int processSize[], int processes)
{
    int allocation[10];
    int i, j;
    for(i = 0; i < processes; i++)
        allocation[i] = -1;
    for(i = 0; i < processes; i++)
    {
        for(j = 0; j < blocks; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }
    printf("\nFIRST FIT Allocation:\n");
    printf("Process No\tProcess Size\tBlock No\n");
    for(i = 0; i < processes; i++)
    {
        printf("%d\t%d\t", i + 1, processSize[i]);
        if(allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void bestFit(int blockSize[], int blocks, int processSize[], int processes)
{

```

```

int allocation[10];
int i, j, bestIdx;
for(i = 0; i < processes; i++)
    allocation[i] = -1;
for(i = 0; i < processes; i++)
{
    bestIdx = -1;
    for(j = 0; j < blocks; j++)
    {
        if(blockSize[j] >= processSize[i])
        {
            if(bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                bestIdx = j;
        }
    }
    if(bestIdx != -1)
    {
        allocation[i] = bestIdx;
        blockSize[bestIdx] -= processSize[i];
    }
}
printf("\nBEST FIT Allocation:\n");
printf("Process No\tProcess Size\tBlock No\n");
for(i = 0; i < processes; i++)
{
    printf("%d\t%d\t", i + 1, processSize[i]);
    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

void worstFit(int blockSize[], int blocks, int processSize[], int processes)
{
    int allocation[10];
    int i, j, worstIdx;

```

```

for(i = 0; i < processes; i++)
    allocation[i] = -1;
for(i = 0; i < processes; i++)
{
    worstIdx = -1;
    for(j = 0; j < blocks; j++)
    {
        if(blockSize[j] >= processSize[i])
        {
            if(worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                worstIdx = j;
        }
    }
    if(worstIdx != -1)
    {
        allocation[i] = worstIdx;
        blockSize[worstIdx] -= processSize[i];
    }
}
printf("\nWORST FIT Allocation:\n");
printf("Process No\tProcess Size\tBlock No\n");
for(i = 0; i < processes; i++)
{
    printf("%d\t%d\t", i + 1, processSize[i]);
    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

int main()
{
    int blocks, processes, i;
    printf("Enter number of memory blocks: ");
    scanf("%d", &blocks);
    int blockSize1[10], blockSize2[10], blockSize3[10];

```

```

printf("Enter sizes of memory blocks:\n");
for(i = 0; i < blocks; i++)
{
    scanf("%d", &blockSize1[i]);
    blockSize2[i] = blockSize1[i];
    blockSize3[i] = blockSize1[i];
}
printf("Enter number of processes: ");
scanf("%d", &processes);
int processSize[10];
printf("Enter sizes of processes:\n");
for(i = 0; i < processes; i++)
{
    scanf("%d", &processSize[i]);
}
firstFit(blockSize1, blocks, processSize, processes);
bestFit(blockSize2, blocks, processSize, processes);
worstFit(blockSize3, blocks, processSize, processes);
return 0;
}

```

```

Enter number of memory blocks: 5
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

FIRST FIT Allocation:
Process No    Process Size    Block No
1             212           2
2             417           5
3             112           2
4             426          Not Allocated

BEST FIT Allocation:
Process No    Process Size    Block No
1             212           4
2             417           2
3             112           3
4             426           5

WORST FIT Allocation:
Process No    Process Size    Block No
1             212           5
2             417           2
3             112           5
4             426          Not Allocated

```

classmate
Date
Page

```

10/10/20 LAB-7
8. WAP to implement memory allocation algorithms
first fit, best fit, worst fit.

#include <stdio.h>
void firstFit (int blockSize[], int blocks, int
processSize[], int processes) {
    int allocation[10];
    int i, j;
    for (i=0; i<processes; i++)
        allocation[i] = -1;
    for (i=0; i<processes; i++)
        for (j=0; j<blocks; j++)
            if (blockSize[j] >= processSize[i])
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
    printf("\nFirst FIT Allocation:\n");
    printf("Process No\tProcess Size\tBlock No\n");
    for (i=0; i<processes; i++)
        printf("%d\t%d\t\t", i, processSize[i]);
        if (allocation[i] != -1)
            printf("\td\t", allocation[i]+1);
        else
            printf("\tNot Allocated\n");
    }

void bestFit (int blockSize[], int blocks, int
processSize[], int processes) {
    int allocation[10];
    int i, j, bestId;
    for (i=0; i<processes; i++)
        allocation[i] = -1;
        bestId = -1;
        for (j=0; j<blocks; j++)
            if (blockSize[j] >= processSize[i])
                if (bestId == -1 || blockSize[j] <
                    blockSize[bestId])
                        bestId = j;
            if (bestId != -1)
                allocation[i] = bestId;
                blockSize[bestId] -= processSize[i];
    printf("\nBest FIT Allocation:\n");
    printf("Process No\tProcess Size\tBlock No\n");
    for (i=0; i<processes; i++)
        printf("%d\t%d\t\t", i, processSize[i]);
        if (allocation[i] != -1)
            printf("\td\t", allocation[i]+1);
        else
            printf("\tNot Allocated\n");
    }

void worstFit (int blockSize[], int blocks, int processSize[],
int processes) {
    int allocation[10];
    int i, j, worstId;
    for (i=0; i<processes; i++)
        allocation[i] = -1;
        worstId = -1;
        for (j=0; j<blocks; j++)
            if (blockSize[j] >= processSize[i])
                if (worstId == -1 || blockSize[j] >
                    blockSize[worstId])
                        worstId = j;
            if (worstId != -1)
                allocation[i] = worstId;
                blockSize[worstId] -= processSize[i];
    printf("\nWorst FIT Allocation:\n");
    printf("Process No\tProcess Size\tBlock No\n");
    for (i=0; i<processes; i++)
        printf("%d\t%d\t\t", i, processSize[i]);
        if (allocation[i] != -1)
            printf("\td\t", allocation[i]+1);
        else
            printf("\tNot Allocated\n");
    }

```

classmate
Date
Page

```

int main() {
    int blocks, processes, i;
    printf("Enter number of memory blocks: ");
    scanf("%d", &blocks);
    int blockSize1[10], blockSize2[10], blockSize3[10];
    printf("Enter sizes of memory blocks:\n");
    for (i=0; i<blocks; i++)
        scanf("%d", &blockSize1[i]);
        blockSize2[i] = blockSize1[i];
        blockSize3[i] = blockSize1[i];
    printf("Enter number of processes: ");
    scanf("%d", &processes);
    int processSize[10];
    printf("Enter sizes of processes:\n");
    for (i=0; i<processes; i++)
        scanf("%d", &processSize[i]);
    firstFit (blockSize1, blocks, processSize, processes);
    bestFit (blockSize2, blocks, processSize, processes);
    worstFit (blockSize3, blocks, processSize, processes);
    return 0;
}

```

classmate
Date
Page

Output

Enter number of memory blocks: 5

Enter sizes of memory blocks:

100 500 200 300 600

Enter number of processes: 4

Enter sizes of processes:

212 417 112 426

FIRST FIT Allocation:

Process No	Process Size	Block No
1	212	2
2	417	5
3	112	2
4	426	Not Allocated

BEST FIT Allocation:

Process No	Process Size	Block No
1	212	4
2	417	2
3	112	3
4	426	5

WORST FIT Allocation:

Process No	Process Size	Block No
1	212	5
2	417	2
3	112	5
4	426	Not Allocated

43 | Page

7. Write a C program to simulate page replacement algorithms.
- a) FIFO
 - b) LRU
 - c) Optimal

```
#include <stdio.h>

void FIFO(int pages[], int n, int frames) {
    int frame[frames];
    int i, j, k = 0, pageFaults = 0, found;
    for (i = 0; i < frames; i++)
        frame[i] = -1;
    printf("\nFIFO Page Replacement:\n");
    for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
                found = 1;
                break;
            }
        }
        if (!found) {
            frame[k] = pages[i];
            k = (k + 1) % frames;
            pageFaults++;
        }
        for (j = 0; j < frames; j++) {
            if (frame[j] != -1)
                printf("%d ", frame[j]);
            else
                printf("- ");
        }
        printf("\n");
    }
    printf("Total Page Faults = %d\n", pageFaults);
}

void LRU(int pages[], int n, int frames) {
    int frame[frames], recent[frames];
```



```

int i, j, pos, found, pageFaults = 0;
for (i = 0; i < frames; i++) {
    frame[i] = -1;
    recent[i] = 0;
}
printf("\nLRU Page Replacement:\n");
for (i = 0; i < n; i++) {
    found = 0;
    for (j = 0; j < frames; j++) {
        if (frame[j] == pages[i]) {
            found = 1;
            recent[j] = i;
            break;
        }
    }
    if (!found) {
        pos = 0;
        for (j = 1; j < frames; j++) {
            if (recent[j] < recent[pos])
                pos = j;
        }
        for (j = 0; j < frames; j++) {
            if (frame[j] == -1) {
                pos = j;
                break;
            }
        }
        frame[pos] = pages[i];
        recent[pos] = i;
        pageFaults++;
    }
    for (j = 0; j < frames; j++) {
        if (frame[j] != -1)
            printf("%d ", frame[j]);
        else
            printf("- ");
    }
}

```

```

    }
    printf("\n");
}
printf("Total Page Faults = %d\n", pageFaults);
}

void Optimal(int pages[], int n, int frames) {
    int frame[frames];
    int i, j, k, found, pageFaults = 0;
    int pos, farthest;
    for (i = 0; i < frames; i++)
        frame[i] = -1;
    printf("\nOptimal Page Replacement:\n");
    for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
                found = 1;
                break;
            }
        }
        if (!found) {
            pageFaults++;
            for (j = 0; j < frames; j++) {
                if (frame[j] == -1) {
                    frame[j] = pages[i];
                    found = 1;
                    break;
                }
            }
            if (!found) {
                farthest = -1;
                pos = -1;
                for (j = 0; j < frames; j++) {
                    int nextUse = n;
                    for (k = i + 1; k < n; k++) {
                        if (frame[j] == pages[k]) {

```

```

        nextUse = k;
        break;
    }
}
if (nextUse > farthest) {
    farthest = nextUse;
    pos = j;
}
}
frame[pos] = pages[i];
}
}
for (j = 0; j < frames; j++) {
    if (frame[j] != -1)
        printf("%d ", frame[j]);
    else
        printf("- ");
}
printf("\n");
}
printf("Total Page Faults = %d\n", pageFaults);
}
int main() {
    int n, frames, i;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    int pages[n];
    printf("Enter page reference string:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &frames);
    FIFO(pages, n, frames);
    LRU(pages, n, frames);
    Optimal(pages, n, frames);
    return 0;
}

```

}

```
Enter number of pages: 13
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2
Enter number of frames: 3
```

FIFO Page Replacement:

```
7 - -
7 0 -
7 0 1
2 0 1
2 0 1
2 3 1
2 3 0
4 3 0
4 2 0
4 2 3
0 2 3
0 2 3
0 2 3
Total Page Faults = 10
```

LRU Page Replacement:

```
7 - -
7 0 -
7 0 1
2 0 1
2 0 1
2 0 3
2 0 3
4 0 3
4 0 2
4 3 2
0 3 2
0 3 2
0 3 2
Total Page Faults = 9
```

Optimal Page Replacement:

```
7 - -
7 0 -
7 0 1
2 0 1
2 0 1
2 0 3
2 0 3
2 4 3
2 4 3
2 4 3
2 0 3
2 0 3
2 0 3
Total Page Faults = 7
```

LAB-10

11/6/26

Q. Write a C program to simulate page replacement algorithms
a) FIFO b) LRU c) Optimal.

```
#include <stdio.h>
void FIFO (int pages[], int n, int frames){
    int fframe[frames];
    int i, j, k=0, pagefaults=0, found;
    for (int i=0; i<n; i++){
        fframe[i]=-1;
        printf("\nFIFO Page Replacement: \n");
        for (i=0; i<n; i++){
            found=0;
            for (j=0; j<frames; j++){
                if (fframe[j]==pages[i]){
                    found=1;
                    break;
                }
            }
            if (!found){
                fframe[k]=pages[i];
                k=(k+1)%frames;
                pagefaults++;
            }
            for (j=0; j<frames; j++){
                if (fframe[j]==-1)
                    printf("%d", fframe[j]);
                else
                    printf("-");
            }
            printf("\nTotal Page Faults = %d\n", pagefaults);
        }
    }
}

void LRU (int pages[], int n, int frames){
    int fframe[frames], recent[frames];
    int i, j, pos, found, pagefaults=0;
```

```
printf("\n");
}
printf("\nTotal Page Faults = %d\n", pagefaults);
}

void Optimal (int pages[], int n, int frames){
    int fframe[frames];
    int i, j, k, found, pagefaults=0;
    int pos, farthest;
    for (i=0; i<frames; i++){
        fframe[i]=-1;
        printf("\nOptimal Page Replacement: \n");
        for (i=0; i<n; i++){
            found=0;
            for (j=0; j<frames; j++){
                if (fframe[j]==pages[i]){
                    found=1;
                    break;
                }
            }
            if (!found){
                pagefaults++;
                for (j=0; j<frames; j++){
                    if (fframe[j]==-1){
                        fframe[j]=pages[i];
                        found=1;
                        break;
                    }
                }
            }
            if (!found){
                farthest=-1;
                pos=-1;
                for (j=0; j<frames; j++){
                    int nextUse=-1;
                    for (k=i+1; k<n; k++){
                        if (fframe[j]==pages[k])
```

```
for (i=0; i<frames; i++){
    fframe[i]=-1;
    recent[i]=0;
}

printf("\nLRU Page Replacement: \n");
for (i=0; i<n; i++){
    found=0;
    for (j=0; j<frames; j++){
        if (fframe[j]==pages[i]){
            found=1;
            recent[j]=i;
            break;
        }
    }
    if (!found){
        pos=-1;
        for (j=0; j<frames; j++){
            if (recent[j]<recent[pos])
                pos=j;
        }
        for (j=0; j<frames; j++){
            if (fframe[j]==-1){
                pos=j;
                break;
            }
        }
        fframe[pos]=pages[i];
        recent[pos]=i;
        pagefaults++;
    }
    for (j=0; j<frames; j++){
        if (fframe[j]==-1)
            printf("%d", fframe[j]);
        else
            printf("-");
    }
}
```

```
nextUse=k;
break;
}
}
if (nextUse > farthest){
    farthest = nextUse;
    pos=j;
}
}
fframe[pos]=pages[i];
}
for (j=0; j<frames; j++){
    if (fframe[j]==-1)
        printf("%d", fframe[j]);
    else
        printf("-");
}
printf("\n");
}
printf("\nTotal Page Faults = %d\n", pagefaults);
}

int main(){
    int n, frames, i;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    int pages[n];
    printf("Enter page reference string: \n");
    for (i=0; i<n; i++){
        scanf("%d", &pages[i]);
    }
    printf("Enter number of frames: ");
    scanf("%d", &frames);
    FIFO (pages, n, frames);
    LRU (pages, n, frames);
    Optimal (pages, n, frames);
}
```

Section 0,

3

⑥

O/p: Enter number of pages: 13

Enter page reference string:

7 0 1 2 0 3 0 4 2 3 0 3 2

Enter number of frames: 3

FIFO Page Replacement:

7 - -

7 0 -

7 0 1

2 0 1

2 0 1

2 3 1

2 3 0

4 3 0

4 2 0

4 2 3

0 2 3

0 2 3

0 2 3

Total Page Faults = 10

LRU Page Replacement:

7 - -

7 0 -

7 0 1

2 0 1

2 0 1

2 0 3

2 0 3

4 0 3

4 0 2

4 3 2

0 3 2

0 3 2

0 3 2

Total Page Faults = 9

Optimal Page Replacement:

7 - -

7 0 -

7 0 1

2 0 1

2 0 1

2 0 3

2 0 3

2 4 3

2 4 3

2 4 3

2 0 3

2 0 3

2 0 3

Total Page Faults = 7